Ders 9 Genişletilmiş Notu: Kod Kusurlarının Devamı ve Kiralama Örneğinin Adım Adım Refactoring'i

Ders 9 Genişletilmiş Notu: Kod Kusurlarının Devamı ve Kiralama Örneğinin Adım Adım Refactoring'i

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Large Class ve Extract Class
- 2. Extract Subclass
- 3. Duplicate Observed Data
- 4. Divergent Change ve Shotgun Surgery
- 5. Move Method ve Move Field
- 6. Primitive Obsession ve "Replace" Eylemleri
- 7. Feature Envy
- 8. Fowler'ın Kiralama Örneğinin Adım Adım Refactoring'i
- 9. Kara Kutu ve Beyaz Kutu Sınaması
- 10. Tip Kodunun Kaldırılması İçin İki Alternatif
- 11. Anti Patterns (Karşıt Kalıplar)
- 12. Örnek Sorular

1. Large Class ve Extract Class

Ders, önceki haftadan kalan yerden, Replace Method with Method Object eyleminin kısa bir hatırlatmasıyla başladı. O eylemde Long Method'u başka yollarla çözemediğimiz durumlarda bütün karmaşıklığı en azından tek bir yerde, yeni bir sınıfta topluyorduk; sonra o sınıf içinde yavaş yavaş çözüyorduk. Ardından kod kusurlarına ve onlara yönelik refactoring eylemlerine devam edildi.

1.1 Large Class (aşırı büyük sınıf)

Bir sınıfın büyük olduğunu görürseniz, bu büyük olasılıkla o sınıfın uyumunun düşük olduğunu gösterir. Başka bir deyişle o sınıfa birbirine yakın olmayan, kimi zaman birbiriyle hiç ilgisiz sorumluluklar atanmıştır. Böyle bir sınıfta üye alan sayısı fazladır; ama hoca, sorunun üye metot sayısından çok metotların çok sayıda satır içermesinde olduğunu söyledi. Büyük olasılıkla üç açıdan da (alan sayısı, metot sayısı, metot uzunluğu) fazlalık vardır; ama yalnız normal tavsiye edilen uzunluğun on katı büyüklükte dört beş metot bile bir sınıfı aşırı büyük hâle getirir.

Çözümler:

- **Extract Class:** bir araya geldiklerinde daha fazla anlam ifade eden üyeleri gruplayıp onlardan bir ya da daha fazla yeni sınıf oluşturmak.
- **Extract Subclass:** oluşturulan sınıfı mevcut sınıfla sahiplik ilişkisine koymak yerine kalıtım ilişkisine koymak. Bazı üye grupları kimi durumlarda birlikte kullanılıp kimi durumlarda kullanılmıyorsa, bunu açıkça bir genelleşme-özelleşme ilişkisiyle ifade etmek daha anlamlıdır. Birden fazla yeni sınıf çıkacaksa kardeş alt sınıflar söz konusu olabilir.
- **Duplicate Observed Data:** bazen büyüklüğün kaynağı, verinin sunumu ile iş mantığının ayrılmamış olmasıdır.

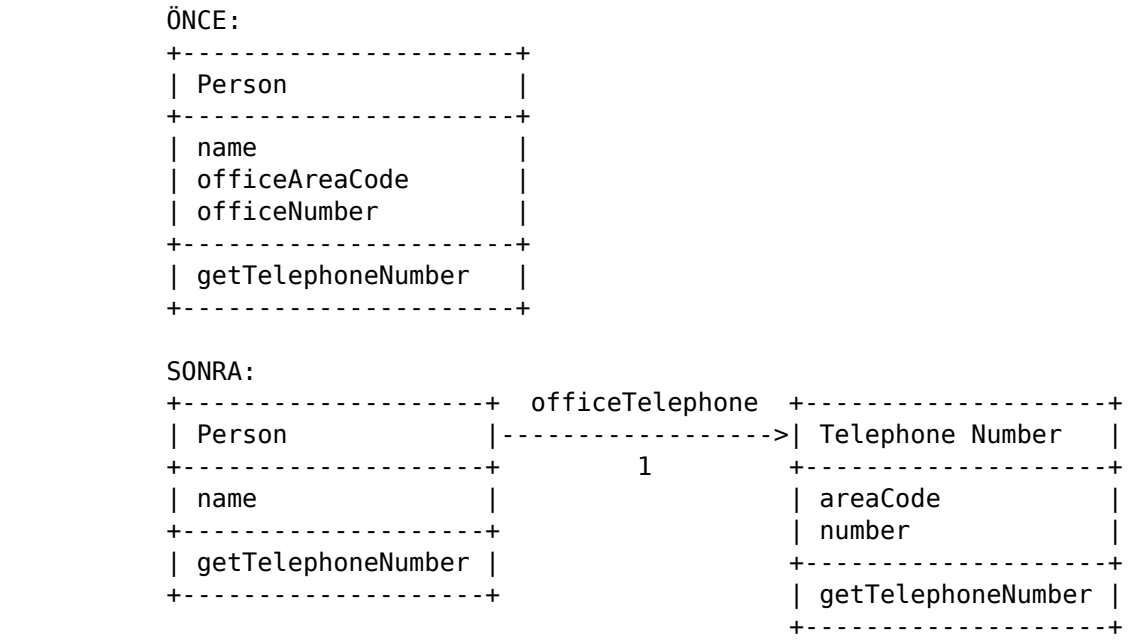
Bu son durumu hoca şöyle açıkladı: kullanıcı arayüzünü oluşturan katman ile iş mantığı katmanı ayrılmaz, ikisi tek bir katman gibi ele alınırsa, iş mantığıyla ilgili her sınıf aynı zamanda kendisinin kullanıcıya gösterilmesi ve kullanıcıdan bilgi alınmasıyla ilgili üyeleri de (formlar, diyalog kutuları, çeşitli arayüz bileşenleri) içerir. Kod en az iki katına çıkar: bir yanda iş mantığına, bir yanda sunum katmanına ait kod. Refactoring'in özü dış arayüzü bozmadan iç yapıyı değiştirmek olduğu için genelde GUI'ye açılmış kısım sabit tutulur, iş mantığıyla ilgili kısım çekilip yeni bir uygulama sınıfı oluşturulur ve eski GUI sınıfı ile yeni iş mantığı sınıfı arasında Observer tasarım kalıbı kurulur. Hoca, tasarım kalıpları daha önce görüldüğü için bu cümleyi artık rahatça kurabildiğini söyledi.

1.2 Extract Class (sınıf çıkartma)

Slayttaki tanım (ekrandan, 00:10:00):

- Birbiri ile ilgisiz iki iş yapan bir sınıfı, bu işlere göre ikiye ayırmaktır.
- İkinci iş ile ilgili üyelerin seçilerek yeni bir sınıfa taşınmasına dayanır.
- Bu yapılandırma da birçok kod kusurunu düzeltir.

Örnek şöyledir:



İlk tasarımda kişinin iletişim bilgileri kişi sınıfının içinde saklanmıştır ve telefon numarası için bir getter vardır. İlk bakışta mantıklı gelebilir. Ama kişinin iş telefonu varsa belki cep telefonu da, ev telefonu da olacaktır. O zaman sınıfa yeni üye alanlar eklemek gerekir. Üstelik bazı kişilerin iş telefonu olup ev telefonu olmayabilir; yine de o alan sınıfta tanımlı durur. Bazı nesneler için gereksiz yere tanımlı alanlar oluşur ve kişi sınıfının kodu daha da karmaşıklaşır.

Telefonla ilgili bilgileri toplayıp yeni bir sınıf oluşturmak bu yüzden en mantıklı çözümdür. Yeni `Telephone Number` sınıfına alan kodu ve numara taşınır; taşınırken üye alanların adı da değiştirilir (`officeAreaCode` → `areaCode`, `officeNumber` → `number`), çünkü yeni sınıf yalnız ofis telefonu için değil, her tür telefon için kullanılabilir. Hoca ilişkinin üzerindeki "1" çokluğu için şunu ekledi: herkesin ofis telefonu yoksa burada "0..1" demek belki daha iyi olurdu; ama sistem bir iş yazılımıysa ve her kişinin iş telefonu mutlaka kaydedilmeliyse "1" yazılabilir.

Bu tasarımda ev ve cep telefonları için aynı sınıftan başka nesneler oluşturulabilir. Yine üç ayrı ilişki alanı gerekir (ev, iş, cep) ve herkesin her telefonu olmayabilir; bu sakınca kalmakla birlikte, her numara için iki alan (alan kodu ve numara) yerine tek bir üye alan tutulmuş olur. Önceki tasarımda kişinin ev telefonu kayıtlı değilse alan koduna ve numaraya hangi değer atanacaktı? Sıfır mı? Sıfır bazı yerlerde santral anlamına gelir. Telefon numaraları nesne olunca, ilgili alanın `null` olup olmadığına bakarak kişinin o türden telefonu olup olmadığı anlamlı biçimde kodlanabilir.

Bir başka yarar: kişi sınıfında telefonla ilgili başka metotlar da birikebilir. Örneğin bir tele-

kom bilgi sisteminde şebeke içi görüşmeler daha düşük, şebeke dışı görüşmeler daha yüksek ücretlendirilir; iki numaranın aynı bölgede ya da aynı operatörde olup olmadığını sınayan kod `Person` sınıfını iyice şişirir. Bu metotların da yeni telefon sınıfına çekilmesi uyumu daha da yükseltir.

1.3 Uyumun üye alan erişim deseniyle hesaplanması

Hoca, uyumun nasıl ölçüldüğünü göstermek için Violet UML Editor'da bir şekil çizdi (ekrandan, 00:16:40). Üstte dört üye alan vardır: `name`, `id`, `officePhoneCode`, `officePhoneNr`. Altta altı metot vardır (m1 … m6). Bir metottan bir üye alana çizilen kesikli ok, o metotta o alanın kullanıldığı anlamına gelir. Kişiyle ilgili metotlar daha çok `name` ve `id` alanlarını, telefonla ilgili metotlar ise telefon alanlarını kullanır.

Uyum, üye alan erişim desenine dayanan metriklerle hesaplanır:

- Verilen: 6 metot, 4 üye alan, çizilen erişim sayısı 10.
- Graf tam bağlı olsaydı, yani her metot her alana erişseydi, erişim sayısı $6 \times 4 = 24$ olurdu.
- Uyum: $10/24 \approx 0{,}42$, yani yaklaşık %42.

Sonra sınıf bir çizgiyle ikiye bölündü: telefon alanları ve onlara erişen metotlar ayrı sınıfa taşındı (Extract Class).

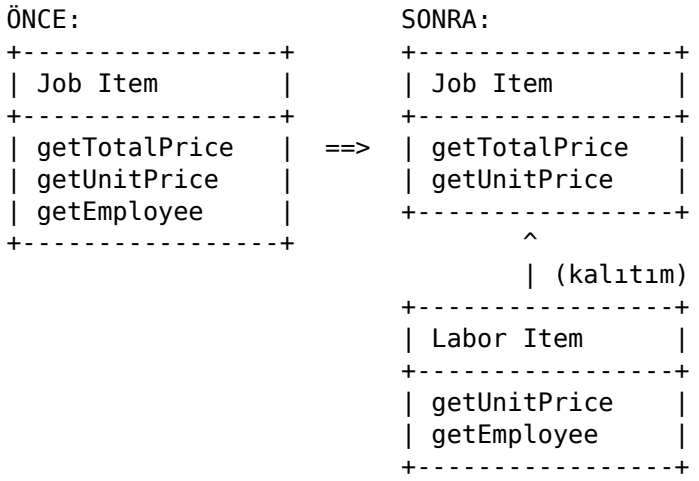
- Birinci sınıf: 4 metot, 2 üye alan; olası erişim $4 \times 2 = 8$, gerçekleşen 6. Uyum $6/8 = 0{,}75$.
- İkinci sınıf: 2 metot, 2 üye alan; olası erişim $2 \times 2 = 4$, dördü de çizilmiş. Uyum $4/4 = 1$, yani %100.

Bir sınıfta 0,75, diğerinde %100 uyum elde edildi; ikisi de başlangıçtaki 0,42'den yüksektir. Hoca, buradan yola çıkarak da iyi bir iş yapıldığının söylenebileceğini belirtti. Metotlar tek tek çok uzun olmasa bile, eski `Person` sınıfı satır sayısı (line of code) açısından da daha uzundu.

> **Ders sonrası not:** Hoca 10/24 oranını bir yerde "bağlaşım" diye andı (00:15:48). Hesaplanan büyüklük sınıf içi uyumdur (cohesion); hoca konuşmanın başında ve sonunda da "uyum" dedi.

2. Extract Subclass

Slayttaki tanım (ekrandan, 00:20:00): bir sınıfın bazı üyeleri her zaman kullanılmıyorsa, bu üyelerden yeni bir alt sınıf oluşturmak daha doğru olacaktır.



`Job Item` yapılacak bir işi temsil eder: işin toplam bedeli, birim fiyatı ve o işi yapan işçi için getter metotları vardır. Genel amaçlı işlerde işi belli bir çalışanın yapıp yapmadığına bakılmaz; toplam fiyat her zaman geçerlidir ve birim fiyata iş masrafı eklenmez. Ama bazı işlerde birim fiyat farklı hesaplanır: çalışanın emeğinin de karşılanması gerekir. Demek ki her durumda çalışanla işlem yapılmıyor; birim fiyatın hesabında bazen farklılık oluyor ve her farklılıkta işi yapan çalışanın bilgisiyle uğraşmak gerekiyor.

"Bazı özel durumlarda" ifadesi genelleşme-özelleşme, yani kalıtım ilişkisine işaret eder. Bu yüzden `Job Item`'dan `Labor Item` alt sınıfı türetilir; `getEmployee` ona taşınır, `getUnitPrice`

ise orada yeniden tanımlanır (override), çünkü çalışanla ilgili bilgi birim fiyat hesabına bu alt sınıfta girer. Genel hâl mevcut sınıfta kalır, özelleşmiş hâl çıkarılan alt sınıfta yer alır.

3. Duplicate Observed Data

Uygulama alanıyla ilgili verinin GUI sınıfında saklanması doğru değildir, ama bazen yapılır. Bunun sonucu, sunum katmanı ile iş mantığı katmanının iç içe geçmesi, ayrımın bozulmasıdır; her şey bir kod yumağına dönüşür. Baştan daha iyi tasarlanmalıydı; ama tasarlanamadı ve belki de kod bu hâliyle canlıya çıktı. Sonraki sürümlerde geriye doğru uyumluluk da korunmak isteniyorsa refactoring gerekir; hoca bunun refactoring'in en önemli motivasyonlarından biri olduğunu hatırlattı.

Örnekte bir `IntervalWindow` sınıfı vardır: belli bir zaman aralığı içinde bir fiyat hesabı, ortalama döviz kuru ya da sıcaklık değişimi gibi bir şeye bakılır. Bu sınıfta hem veri girişi için üye alanlar, hem odağın alınması ya da kaybedilmesi gibi GUI olaylarını işleyen kod, hem de aralığın uzunluğunu hesaplayan iş mantığı metotları vardır. Yani sınıf aynı anda hem iş mantığını hem GUI olaylarını yürütmeye çalışmaktadır.

Çözüm, sınıfı ikiye bölmektir: iş mantığıyla ilgili kısım yeni bir sınıf (`Interval`) olur. GUI sınıfı silinmez, ama iş mantığına ait alan ve metotlar ondan çıkarılır. İlişki de tamamen koparılmaz: GUI sınıfında bir olay (örneğin veri girişi) gerçekleştiğinde iş mantığı sınıfındaki nesnelerin haberdar edilmesi ve güncellenmesi gerekebilir; bazı olaylarda da yalnız başka bir zaman aralığındaki durumun gösterilmesi gerekebilir. Her çağrıyı doğrudan GUI'den iş mantığı sınıfına aktarmak yerine Observer kalıbı kurulabilir.

Hoca burada iki senaryoyu ayırdı:

- GUI penceresi veri girişi için değil raporlama içindir; gözlenen varlığın durumundaki değişiklikleri gösterir (örneğin bir borsa endeksini izleme). Bu durumda gözlenen (subject) iş mantığı sınıfıdır, gözlemleyen (observer) GUI bileşenidir. Slayttaki şema bu senaryoya uygundur.
- Olaylar GUI'de oluşur ve iş mantığı sınıfına aktarılır. Hoca örneği önce bu yönde düşündüğünü söyledi.

Her iki durumda da iş mantığıyla sunum mantığının iç içe geçtiği bir yapı bölünür. Hoca sonucu şöyle bağladı: bir varlık değişince o değişimi göstermek gibi bir GUI sorunu varsa, yapılan bölme işleminin adı **Duplicate Observed Data**'dır; öbür türlü bir durum söz konusuysa yapılan iş normal bir **Extract Class**'tır. İki eylem bu açıdan birbirine benzer; fark buradadır.

> **Ders sonrası not:** Hoca bu kısımda bir an "normal Extract Method" dedi, hemen ardından "Extract Class demek daha doğru" diye düzeltti (00:26:34).

4. Divergent Change ve Shotgun Surgery

Bu iki kod kusuru birbirine çok benzer. Özleri şudur: bir yazılım hatası ya da gereksinim değişikliği söz konusu olduğunda, gerekli düzeltmeler için birden fazla yere dokunmak gerekiyorsa bu iki kusurdan biri vardır. Fark şudur:

- Değişiklikler **aynı sınıfın farklı metotlarında** yapılıyorsa: **Divergent Change**.
- Değişiklikler **birden fazla sınıfın az sayıda metodunda** (birer, en fazla ikişer üçer) yapılıyorsa: **Shotgun Surgery**.

Hoca Shotgun Surgery adını bir benzetmeyle açıkladı. Çifte (av tüfeği) kartuşunun içinde küçük küçük saçma taneleri vardır; ateş edildiğinde bu parçalar yayılır ve yakın mesafeden ateş edilen kâğıt, metal ya da plastik bir yüzeyi delik deşik eder. Türkçede bu tanelere "saçma" denmesi de "anlamsız" anlamında değil, "saçılmış" anlamındandır. Violet'te çizdiği örnek bir tasarım üzerinde beş ayrı yeri işaretleyince değişikliklerin tasarımın üzerine saçıldığı görüldü: bir sınıfın iki metodu, başka bir sınıfın bir metodu, bir başkasının bir metodu… İşte buna Shotgun Surgery denir. Bir sınıfın bir dolu metodunda düzeltme yapılıyorsa ona Divergent Change denir. Hoca ayrıca "pire için yorgan yakmak" deyimini andı: bir sorun için her şeyi kökten değiştirmek etkili bir çözüm değildir.

Çözümler:

- **Extract Class:** Shotgun Surgery söz konusuysa, farklı sınıflardaki bu metotlar şu an ayrı yerlerde dursalar da aslında bir arada daha fazla anlam taşıyorlardır. Bu metotlardan ve onların eriştiği üyelerden yeni bir ya da birden fazla sınıf çıkarılır ve sahiplik ilişkisiyle kod sürdürülür.
- **Move Field / Move Method:** bu üyeler mevcut sınıflardan birine toplanabilir. Bir üye alan taşınıyorsa Move Field, bir üye metot taşınıyorsa Move Method yapılmış olur.

5. Move Method ve Move Field

Bir metodun kendi tanımlandığı sınıftan çok başka sınıflardan çağrıldığı görülüyorsa, o metot yanlış yerde tanımlanmıştır. Metodu, onun çağrılmasına en çok ihtiyaç duyan sınıfa taşımak en doğrusudur; çağıranlar farklı sınıflardansa, en çok ihtiyaç duyana taşınır.

Bu iki şekilde yapılabilir:

- Metot eski sınıfından tamamen silinip yeni sınıfa taşınır. Bu tercih edilmez, çünkü metot silinince eski sınıfın arayüzü değişir.
- Eski yerinde yalnız metot imzası kalır; eski sınıftaki metot yalnızca mesajı yeni sınıftaki nesnenin metoduna iletir. Buna **delegation** denir; bir tür proxy gibi de düşünülebilir.

Move Method başka kod kusurlarını düzeltmeye yönelik bir eylemdir; ayrıca daha karmaşık refactoring eylemlerinin ilk adımı olarak da düşünülebilir. Örneğin Extract Class yaparken üye alanları ve metotları taşımak gerekir; bu taşıma, ilişki koparılmadan, arada delegation kurularak yapılabilir.

Hocanın soyut örneği: iki sınıf vardır. Sınıf 1'deki bir metot, kendi sınıfında yalnız tek bir başka metottan çağrılırken, Sınıf 2'nin beş altı ayrı metodu tarafından çağrılmaktadır. Demek ki bu metodun yeri Sınıf 1 değil, Sınıf 2'dir; oraya taşınır. Sınıf 1'de de metot kalır, ama yaptığı tek şey Sınıf 2'deki metodu çağırmaktır. Bu iki sınıf birbirinden kopuk da değildir: metot Sınıf 2'den çağrılabildiğine göre arada bir sahiplik ya da parça-bütün ilişkisi vardır; o ilişki ters yönde kullanılarak Sınıf 1'e gelen çağrı Sınıf 2'ye yönlendirilir.

6. Primitive Obsession ve "Replace" Eylemleri

6.1 Primitive Obsession (ilkellere takıntı)

Hoca bu kusurun adının biraz esprili olduğunu söyledi: ilkel tiplere takıntı, ya da ilkel tiplerin aşırı kullanımı. Yapısal programlamada deneyimi çok olup nesneye yönelimi yeni öğrenenlerde ilkel tipleri aşırı yoğun kullanma eğilimi olabilir. Bu kusurun farklı biçimleri vardır; hoca her birini, onu düzelten refactoring eylemi üzerinden anlatmayı seçti. Hepsinin adı "Replace" ile başlar:

- **Replace Data Value with Object:** ilkel tiplerden oluşturulmuş bir veri değeri yerine bir nesne kullan.
- **Replace Type Code with ...:** tip kodu (kan grubu, çalışan türü gibi) yerine bazen ayrı bir sınıf, bazen alt sınıflar, bazen de State ya da Strategy kalıbı kullan.
- **Replace Array with Object:** ilkel değerler içeren dizilerle yapılan tasarımlar yerine nesne kullan. Yapısal programlamada bunu, başka şansımız olmadığı için yaparız; struct yapısı da onun yerine kullanılabilir.

Hoca, bugün görülen Extract Class ve önceki hafta görülen Introduce Parameter Object işin başında kullanılsaydı belki bu noktaya hiç gelinmeyeceğini de ekledi.

6.2 Replace Data Value with Object

Slayttaki tanım (ekrandan, 00:40:00): bir üye alan, hele ki bu alan ile ilgili çeşitli davranışlar söz konusu ise, bir sınıf ile simgelenmelidir.

```
ÖNCE:   +------------------+
        | Order            |
        +------------------+
        | customer: String |
        +------------------+
```



Sipariş sınıfında siparişi veren müşterinin adı bir `String` üye alan olarak tutulmaktadır. Hoca bunun sakıncalarını saydı. Birincisi, müşteri adı bazen büyük harfle, bazen küçük harfle girilebilir. İkincisi, ad tekil bir tanımlayıcı değildir. Hoca kendi deneyimini anlattı: çocukken iki adı ve bir soyadı olduğu için adının benzersiz olduğunu sanırmış; sonra bir 23 Nisan kutlamasında televizyonda, başka bir okuldan aynı adı taşıyan başka bir çocuğun şiir okuyacağı anons edilmiş. Aynı ad ve soyadlı farklı müşteriler olabilir. Bu yüzden siparişte müşterinin yalnız adını üye alan olarak tutmak doğru bir modelleme değildir; mutlaka bir `Customer` sınıfı oluşturulmalı, ad onun üye alanı olmalı ve siparişin müşteriyle ilişkisi kurulmalıdır.

Hoca, Fowler'ın ilişkiyi bu biçimde (parça-bütün) kurduğunu, ama başka türlü de düşünülebileceğini söyledi; önemli olanın müşterinin sipariş içinde bir üye alan olarak kalmayıp başlı başına bir varlık olarak temsil edilmesi olduğunu vurguladı. Slaytın devamı:

- Programın tümü incelendiğinde, belli bir müşterinin tüm siparişlerinin aranması gibi işlemler yapıldığı görülecektir.
- Bu işlemler çoğaldıkça, Müşteri sınıfı ayrılmadığı sürece Sipariş sınıfı aşırı büyüyecektir (large, hatta "huge" class).
- İyisi mi şimdiden bu sınıfları ayıralım.

6.3 Tip kodları ve Replace Type Code with Class

Tip kodları genelde yapısal programlamadaki sabit değişken kullanma alışkanlığından gelir. Örnek: kişinin kan grubu saklanmak isteniyor ve tam sayı olarak tutuluyor. Bir kişinin kan grubu 0, A, B ya da AB olabilir; beşinci bir grup yoktur. Bu dört tür `public final static` sabit olarak tanımlanır ve her birine bir değer atanır: örneğin O'ya 0, A'ya 1, B'ye 2, AB'ye 3. Kişinin kan grubu alanındaki değer 1 ise kişi A grubudur.

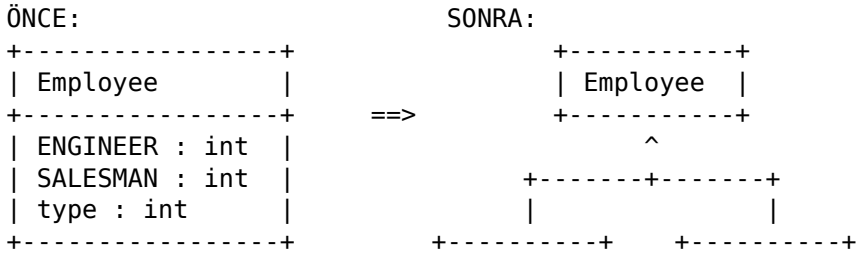
Hoca bunun doğru olmadığını söyledi ve temel ilkelerden birini hatırlattı: değişen ne ise onu bul ve sarmala. Bu yüzden `BloodGroup` diye yeni bir sınıf oluşturulmalı, kan grubunun çeşitli durumları ona taşınmalıdır. Kişi sınıfında kan grubu üyesi kalır ama türü artık `int` değil `BloodGroup` olur; o alana hangi nesne atanmışsa kişinin kan grubu odur.

Birçok programlama dilinde `enum` yapısı vardır; enum kullanılsa bu hataya hiç düşülmez ve enum kullanmanın hiçbir sakıncası yoktur. Hoca parantez içinde şunu ekledi: Java'da enum'lar da aslında bir tür sınıf olarak ele alınır; enum'un her olası değeri o türden bir nesnedir ve enum'un kurucusu, alanları, hatta metotları olabilir. Özetle bir tip kodu varsa ya enum kullanılır ya da dil enum desteklemiyorsa ona benzer bir tip sınıfı yazılır.

> **Ek bilgi (derste anlatılmadı):** Bu eylemin Fowler'ın kataloğundaki adı Replace Type Code with Class'tır. Hoca derste eylemi adıyla anmadı, yalnız "tip kodunu farklı bir sınıfla değiştirmek" diye tanıttı.

6.4 Replace Type Code with Subclasses

Slayttaki tanım (ekrandan, 00:50:00): bir sınıfın davranışını belirleyen bir ilkel tip kodu yerine kalıtımın kullanımı daha uygun olacaktır.





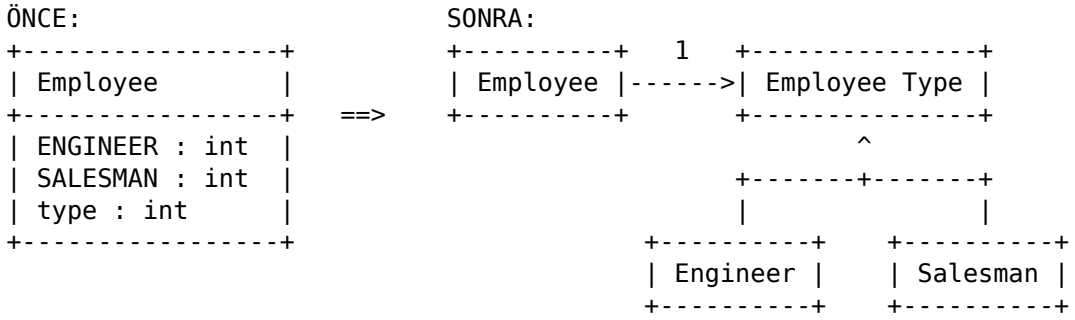
Şirkette mühendisler, satış elemanları, yöneticiler gibi farklı türden çalışanlar vardır. Bu türler `public final static` sabitlerle tanımlanır ve çalışan sınıfında bir `type` alanı tutulur: 0 mühendis, 1 satış elemanı, 2 müdür… Çalışanlara bir iş yaptırılacağında, maaşları ya da özlük hakları hesaplanacağında, bütün o metotlarda bir switch-case görülür: "switch type; case ENGINEER: şöyle yap; case SALESMAN: böyle yap." Bu, Long Method'a yol açar; bir dolu özlük hakkı için bir dolu farklı metot olacağından bu uzun metotlar bir araya gelip Large Class oluşturur.

> **Hocanın vurgusu:** Kendinizi bir switch-case kullanırken ya da if-else-if zinciri kurarken görüyorsanız büyük olasılıkla yanlış bir modelleme yapmışsınızdır. Bu işi kalıtım ve çok biçimlilik üzerinden yapmak en doğrusudur.

Çözüm: `Employee` sınıfından `Engineer` ve `Salesman` alt sınıfları oluşturulur; özlük metotlarındaki her case kısmı eski `Employee` sınıfından çıkarılıp ilgili alt sınıfa taşınır. Böylece daha derli toplu sınıflar elde edilir. Türü olmayan bir çalışan söz konusu olabiliyorsa `Employee` soyut yapılmaz; ama her çalışan mutlaka belli bir türde olmalıysa `Employee` soyut yapılır ve ondan hiç nesne oluşturulmaz.

6.5 Replace Type Code with State/Strategy

Slayttaki tanım (ekrandan, 00:47:20): önceki örnekte kalıtım kullanımı bir nedenle mümkün değilse, State veya Strategy tasarım kalıbı da kullanılabilir.



Hoca kalıtımın neden kullanılamayabileceğine dair bir örnek bulmaya çalıştı (örneğin `Employee` zaten başka bir sınıftan türemiş olabilir), ama bunun alt sınıf çözümünü bozmadığını görünce örneği bıraktı. Asıl gerekçe şudur: `Employee` sınıfının eski hâli yerine bir `Employee Type` arayüzü (ya da soyut sınıfı) üzerinden gidilir. Mühendis, satış elemanı gibi her türün özlük hakkı hesapları için gereken bütün metotların imzaları arayüzde tanımlanır, gövdeleri onu gerçekleyen sınıflarda yazılır. Eski hâldeki uzun case'lerin her biri ilgili stratejiye atanır.

Bu tasarımın alt sınıf çözümüne üstünlüğü şudur: bir kişi mühendisken satış kadrosuna, satış kadrosundayken yönetici kadrosuna alınabilir. Hocanın "kötü" dediği ilk tasarımda (tip kodu) bile bu esneklik vardı: `type` alanının değeri değiştirilince çalışan hemen başka türden bir çalışan olur. Alt sınıf çözümünde ise bu esneklik yoktur, çünkü kalıtım sınıf düzeyinde tanımlanan bir ilişkidir ve bir nesnenin türü sonradan değiştirilemez. Bir nesne çok biçimlilik gereği üst sınıfın beklendiği yerde kullanılabilir ama kardeş sınıfa dönüştürülemez. Bir mühendisi satış elemanı yapmak için mühendis nesnesinin bütün verisini kopyalayıp yeni bir `Salesman` nesnesi oluşturmak, eski kaydı silip yerine yenisini koymak gerekir. Bunun için `Engineer` sınıfına bir (belki birden fazla) copy constructor ya da `Company` sınıfına `promote`/`changeType` gibi bir metot eklenmesi gerekir; epey uzun bir iştir.

Strateji çözümünde ise her çalışanın mutlaka bir `Employee Type` nesnesi vardır. Bir mühendisi satış elemanı yapmak için bu üyeye bir `Salesman` nesnesi atamak yeterlidir; değişiklik anında devreye girer. `Employee` nesnesi sistemde nerede saklanıyorsa orada, kimliği ve nesne adresi değişmeden yaşamaya devam eder. Değişen kısım bulunup sarmalanmış, değişiklik yalnız o kısma aktarılmış olur.

> **Hocanın vurgusu:** Çalışanın türü çalışma sırasında değişebiliyorsa (terfi, kadro değişikliği) ortaya çıkan yapıya State kalıbı demek doğru olur. Tür değişmiyor, yalnız

özlük hakları türe göre farklı hesaplanıyorsa bu yapı Strategy kalıbıdır. Arada küçük ama önemli bir nüans vardır.

6.6 Replace Array with Object

Hoca bu örneği daha önce de sorduğunu, yapısal programlama açısından bakıldığında doğru göründüğünü söyledi. Ligdeki futbol takımlarının adları, puanları ve averajları tutulmak isteniyor. Struct bilinmiyor ya da dil desteklemiyorsa bunlar üç ayrı tek boyutlu dizide tutulabilir: takım adları için bir `String` dizisi, puanlar için bir `int` dizisi, averajlar için bir dizi. Her dizide aynı indisteki bilgiler aynı takıma aittir. Hoca Java üzerinden kısa bir kod yazdı (Java'yı seçti, çünkü C'de karakter dizilerinin dizisi iki boyutlu matris gerektirir).

Bu yaklaşımda Galatasaray'la ilgili verilerle uğraşılacaksa hep üç dizinin 0 indisindeki değerlerle, Fenerbahçe ile ilgili verilerle uğraşılacaksa hep 1 indisindeki değerlerle çalışılır. Yapı hataya açıktır; hoca canlı yazarken de bir yanlış yaptı ve bunu örnek gösterdi. Hoca, slayttaki örnekte verileri iki boyutlu bir dizide de tutmuş olabileceğini söyledi; o durumda sayılar da dizgi olarak saklanır ve aritmetik işlem için `Integer` sınıfının dönüştürme metotlarıyla tam sayıya çevrilmeleri gerekir. İş giderek çetrefilleşir.

Struct dizisi olsaydı sorun çözülürdü; nesneye dayalı tasarımda ise bir `Takım` sınıfı oluşturulmalı, üye alanları tasarlanmalı ve değerler atanmalıdır. Daha önce üç ayrı dizide yapılan işlemler tek bir nesne dizisi üzerinden, tek bir nesnenin üye alanlarıyla yapılır. Hoca bunun en doğal ve en doğru yol olduğunu söyledi.

7. Feature Envy

Aradan sonra ders yeni bir kod kusuruyla sürdü. Slayttaki metin (ekrandan, 01:10:00):

- Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa, bu duruma "özellik kıskançlığı" adı verilir.
  - "Çok" sıfatı ile o koddaki ortalama (veya %20 gibi bir eşikten fazla) farklı sınıf üyesi erişim sayısının üzerinde bir sayı ifade edilmektedir.
  - Sorumlulukların doğru atanmadığının göstergesi olabilir.
  - Elbette bağlaşım ilkesi nedeniyle başka sınıflardan nesnelere erişmek gerekecektir; bu nedenle hiçbir sınıf sadece kendi metotlarını kullanmaz. Öyle olsaydı bu kez de uyum ilkesini olumsuz yönde etkilemiş olurduk.
  - Strategy ve Visitor kalıpları bir özellik kıskançlığı durumu olarak yorumlanmamalıdır.
- Çözüm için olası refactoring eylemleri:
  - Move Method.
  - Extract Method: eğer kıskançlık metodun sadece bir kısmında ise o kısmı çıkartıp kıskanılan üyelerin bulunduğu sınıfa taşınabilir.

Hoca ayrıca tersini de aynı kusur saydı: bir metot ait olduğu sınıftan çok başka sınıflar tarafından kullanılıyorsa da aynı durum vardır. Örneğin A sınıfının M1 metodu, A'nın diğer metotlarından çok B ve C sınıflarının çeşitli metotları tarafından kullanılıyorsa, çağrılma sıklığına bakarak M1'i B'ye ya da C'ye koymak daha doğru olabilir.

"Çok" göreli bir kavramdır: neye göre, kime göre? Burada ortalama bir eşiğin üstü kastedilir. Bu eşik için daha önce yapılmış ve belli bir kalitenin üstünde olduğu bilinen projelerdeki ortalama sayı kullanılabilir; literatürde farklı problem alanlarında ve farklı dillerde yazılmış yazılımlarda bu incelemeyi yapan çalışmalardan da alınabilir; ya da o anda yazılan projenin kendi ortalaması hesaplanıp örneğin ortalamanın üçte bir fazlası "çok" sayılabilir.

Böyle bir durumun anlamı genelde şudur: sorumluluklar doğru atanmamıştır; başka bir deyişle üye alan ya da metot, ait olması gereken yerde tanımlanmamıştır. Sıfır bağlaşım diye bir şey yoktur; ilişkide olunan diğer sınıfların metotlarını çağırmak doğaldır. Sorun, belirli bir metodun dengesiz biçimde başka sınıflara bağımlı olmasıdır. Çözüm büyük olasılıkla Move Method'dur; metot ilk yerinden tamamen silinmez, orada tek satırlık bir delegation çağrısı tutulur. Kıskançlık metodun belli bir kısmına odaklanmışsa yalnız o kısım çıkarılır ve kıskanılan sınıflardan ona en çok ihtiyaç duyana taşınır.

Hoca bu kusurla birlikte ders kapsamında planlanan bütün kod kusurlarının görüldüğünü söyledi.

8. Fowler'ın Kiralama Örneğinin Adım Adım Refactoring'i

8.1 Yaklaşım

Slaytın son kısmı (ekrandan, 01:10:00): şimdiye kadar incelenen düzenlemelerden bazılarının birlikte kullanılarak, yapısal programlama yaklaşımı izleri taşıyan bir tasarımın nasıl değiştirildiğinin bir örneği için Fowler'ın *Refactoring* kitabının 1. bölümü incelenebilir.

Hoca, Fowler'ın kitabında bu bütüncül örneği en başa koyup kod kusurlarını ve refactoring eylemlerini sonra anlattığını; kendisinin ise önce parçaları, sonra bütünü öğretmeyi tercih ettiğini söyledi. Aynı adımları her seferinde aynı sırayla yapmak zor olduğu, bir gün bir yerden öbür gün başka yerden başlanabildiği için tutarlı bir senaryo izlemek amacıyla notlarına bakarak ilerledi.

Kod Eclipse'te düzenlenmiştir (ekrandan, 01:40:00): `refactoring` klasöründe `fowler_00` (kodun özgün hâli), `fowler_01`, … `fowler_10`, `fowler_10_alt`, `fowler_11_alt` paketleri; `testing` klasöründe aynı adlı paketlerde `TestAll`, `TestCustomer`, `TestMovie` sınıfları vardır. Özgün hâlin testleri önceki derste çalıştırılmış ve hepsinin geçtiği gösterilmişti.

Çalışma yöntemi şudur: bir önceki paket kopyalanıp yeni adla yapıştırılır, değişiklik yeni pakette yapılır, test sınıfları da kopyalanır ama **test kodu değiştirilmez**. İş mantığı kodu değiştirildiği hâlde test kodunu değiştirmek gerekmiyorsa, kodun dışarıya verdiği hizmet arayüzü değişmemiş demektir; yani refactoring özüne uygun yapılmıştır. Değişiklikten vazgeçmek istenirse yeni paket silinip eskisinden devam edilir. İki paketteki aynı sınıf, Ctrl ile seçilip sağ tık → Compare With → Each Other ile karşılaştırılır. Hoca bir adımı canlı yaptı (JUnit kütüphanesini Build Path → Add Library ile projeye ekleyerek); sonraki adımlarda zamanı verimli kullanmak için yalnız neyin değiştiğini söyleyip ilgili paketteki kodu gösterdi.

Hoca, Eclipse'in bazı refactoring eylemleri için otomatik seçenek sunduğunu ama bunların her zaman doğru çalışmayabildiğini söyledi; yeni sürümlerde düzelmiş olabilir, ama en sağlamı eylemleri kendiniz yapmaktır. Konuyu öğrenmek için de adımları elle yapmak gerekir.

8.2 İlk tespitler

`Customer` sınıfının `statement()` metodu en uzun ve karmaşık koddur; "ilk neşter" oraya vurulacaktır. Hocanın gözüne çarpanlar:

- `statement()` çok uzundur (Long Method); bölünmelidir.
- Uzunluğu en çok artıran kısım switch-case'tir. Bu bir yandan Long Method, öbür yandan tip kodundan kaynaklanır: filmin üç türü birer tip kodudur ve hepsi bir switch-case içinde ele alınır. Tip kodları ortadan kaldırılacaktır.
- Geçici değişkenler kodu uzatmaya meyillidir. Kümülatif toplam kaldırılamaz ama en azından `thisAmount` kaldırılabilir.
- `statement()`, her kira kaydı üzerinden filmin türüne erişir. Sınıf şemasına bakıldığında `Customer`, `Movie` ile yalnız `Rental` üzerinden ilişkilidir; buradan `Movie`'ye erişmek dolaylı ilişkinin aşırı kullanımı ya da Feature Envy gibi görülebilir.

Bunlar düzeltildikçe yeni kusurlar görülecek ve onlar için yeni eylemler yapılacaktır. Hoca, çok biçimlilik gerektiği açık olsa da bütün tasarımı sıfırdan yapmayacağını, küçük adımlarla refactor edeceğini vurguladı.

8.3 Adım 1: switch bloğunun çıkarılması (Extract Method)

Slayttaki özet (ekrandan, 01:20:00):

- İlk adım: aşırı uzun olan `statement` metodunu parçalamak.
  - `statement` metodunun da en uzun parçası switch bloğu olarak gözüküyor.
  - Aslında switch kullanımı çok biçimliliğe ihtiyaç duyulduğunun göstergesidir.
  - Ancak değişiklikleri küçük adımlarla yapmamız, hata yapmamız hâlinde hatamızı düzeltmeyi kolaylaştıracaktır.
  - Sonuç olarak bu adımda switch bloğu, geçici değişkenlere de dikkat ederek, `amountFor` adlı bir metoda taşınmıştır. Taşınacak alanda iki yerel değişkene erişim yapıldığı

için Eclipse düşünülen düzenlemeyi gerçekleştirememiştir; yerel değişkenler `thisAmount` ve `each` tanımlama sırası değiştirilince Eclipse de `extractMethod` işleminde başarılı oldu.
- Sonuç kod: `refactoring/fowler01` dizininde.
- Test case'leri yeniden çalıştırmayı unutmayalım. Sadece `Customer` sınıfında değişiklik yapıldığı için `TestCustomer` birim testini çalıştırmak yeter (`testing/fowler01`). Aslında test kodu değişmedi; ancak `Customer` sınıfının yeni hâli `fowler01` paketine alındığı için `TestCustomer` da bu yeni pakete kopyalandı.

Switch bloğunun ihtiyacı şudur: o anki kira kaydını bilmek, ondan filmi ve filmin fiyat kodunu, ayrıca kaç gün kiralandığını öğrenmek. Sonuçta o kiralamadan doğan bedeli hesaplar. Bu yüzden blok kesilip `amountFor` adlı yeni bir metoda konur ve ona parametre olarak o anki kira kaydı (`each`) verilir. Metodun dönüş türü `double`'dır, çünkü bedel kuruşlu olabilir. `statement()` içinde switch'in olduğu yerde artık iki satır kalır: bedel bu metottan alınır.

Yeni metot **private** tanımlandı. Hoca gerekçesini şöyle açıkladı: tek bir kira kaydı için bedelin dışarıdan doğrudan çağrılmasına gerek yoktur, anlamı da yoktur; müşteri neyi kiraladığını kendisi bilir. Dışarıdan "şunu kiralasaydı ne olurdu" gibi bir senaryo hesabı için yararlı olabilir ama bu çok kısıtlı bir yarardır. Bu hesap sınıfın iç çalışma mantığına aittir.

Testler çalıştırıldı: derleme hatası yok, bütün testler geçti.

8.4 Adım 2: yeniden adlandırma

Yeni `amountFor` metoduna bakınca, bu hâliyle anlamsız kalan değişken adları görülür. Parametrenin adı `each` ("her biri"), ama metot her bir kira kaydı için değil, kendisine verilen tek bir kayıt için çalışır. Bu yüzden parametrenin adı `aRental` yapılır; metot genel amaçlı bir bedel hesaplama metodu olduğu için yerel değişkenin adı `thisAmount` yerine `result` olur. `statement()` içindeki geçici nesnenin adı da `each` yerine `aRental` yapıldı.

Ad değiştirmeyi Eclipse üzerinden yapmak kolay ve verimlidir; hoca bunun ilk zamanlardan beri doğru çalışan bir özellik olduğunu söyledi: değişkenin, üye alanın, metodun, hatta paketin ya da sınıfın üzerinde sağ tık → Refactor → Rename denip yeni ad verilince değişiklik her yerde tekrarlanır.

> **Hocanın vurgusu:** "Hocam, buna da mı refactoring eylemi diyorsunuz?" denebilir. Anlaşılabilirliği artırmaya yönelik bir değişiklik olduğu için, küçük de olsa küçümsemeden bu bir refactoring eylemidir.

Yalnız ad değiştirmek testleri bozmaz. Ama örneğin getter yerine doğrudan alana erişim açılmış olsaydı test kodu geçersiz hâle gelip derleme hatası verebilirdi. Eclipse'in Rename işlemi bütün bağımlı yerleri de değiştirdiği için bu sorun çıkmaz; yine de adet olduğu üzere testler yeniden çalıştırılıp yeşil çubuk görülür.

8.5 Adım 3: `amountFor`'un `Rental`'a taşınması (Move Method)

`amountFor` parametre olarak bir kira kaydı alır, o kaydın filmini ve kiralanma gün sayısını kullanır. Arada bir bağlaşım vardır ve metot bunun üzerinden çağrı yapar; ama `Customer` sınıfının metodu olduğu hâlde `Customer`'la ilgili **hiçbir bilgiye erişmez**: yalnız geçici değişken ve metot parametresi vardır; Eclipse'te mavi renkle, kâğıtta alt çizgiyle başlayan adlarla görünen tek bir üye alan bile yoktur. Metot burada eğreti durmaktadır ve daha çok `Rental`'ın üyelerine ihtiyaç duyar (Feature Envy).

Bu yüzden metot `Rental` sınıfına taşınır ve adı da değiştirilir: metodun yaptığı iş bir kiralamadan doğan bedeli hesaplamak olduğuna göre, kiralamayı simgeleyen nesnede "bedel öğrenme" metodu olarak adlandırılması daha doğrudur: `getCharge()`. `Customer` artık bedeli `Rental`'a sorar. Yani önce Extract Method yapılmış, şimdi Move Method yapılmıştır.

`fowler_02` ile `fowler_03` karşılaştırıldığında: `Customer` eskiden aynı sınıftaki `amountFor`'u kira kaydını parametre vererek çağırıyordu; şimdi listeden sıradaki kaydı aldıktan sonra doğrudan ona "kiralama bedelin nedir?" diye sorar. Private `amountFor` metodu `Customer`'dan kalkmıştır. `Rental`'a `getCharge()` eklenmiştir (ekrandan, 01:40:00):

```
double getCharge( ) {
    double result = 0;
    //determine amounts for each line
    switch (getMovie().getPriceCode()) {
    case Movie.REGULAR:
        result += 2;
        if (getDaysRented() > 2)
            result += (getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
    ...
```

Hoca, metodun yeni yerinde daha sade göründüğünü söyledi: artık ekstradan kira kaydının getter'larını çağırması gerekmiyor; `getDaysRented()` kendi metodudur. `Movie`'nin yalnız fiyat koduna erişir. Hem kendi üye alanına hem sahip olduğu nesnenin bir alanına erişen bu dağılım dengelidir; üye alan erişimine bakıldığında daha homojen bir dağılım görülür ve metodun yeri konusunda göze batan bir şey kalmaz. Testler yeniden çalıştırıldı ve geçti.

8.6 Ara konu: kara kutu ve beyaz kutu

Bu adımdan sonra hoca, `Rental.getCharge()` için yeni bir test yazılıp yazılmayacağını tartışmak üzere sınama türlerine kısaca değindi; ayrıntısı 9. bölümdedir. Kararı şuydu: mevcut testlerle, yani kara kutu yaklaşımıyla devam etmek.

8.7 Adım 4: `thisAmount` geçici değişkeninin kaldırılması

`statement()` yarı yarıya kısalmıştır ama hâlâ geçici değişkenler vardır. `totalAmount` kaldırılamaz: kümülatif toplamdır ve biriktirileceği bir yer olmalıdır. `thisAmount` ise kaldırılabilir. Eclipse'te üzerine tıklayıp vurgulayınca (ekrandan, 01:50:00) görülür ki ona bir kez değer atanmış (`thisAmount = aRental.getCharge();`), sonra hiç değiştirilmeden doğrudan kullanılmış ve `totalAmount`'a eklenmiştir:

```
public String statement( ) {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Enumeration<Rental> rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";
    while (rentals.hasMoreElements()) {
        double thisAmount = 0;
        Rental aRental = (Rental) rentals.nextElement();
        thisAmount = aRental.getCharge();
        // add frequent renter points
        frequentRenterPoints ++;
        // add bonus for a two day new release rental
        if ((aRental.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                && aRental.getDaysRented() > 1) frequentRenterPoints ++;
        //show figures for this rental
        ...
```

`thisAmount`'un geçtiği her yere `aRental.getCharge()` yazılınca değişken tamamen ortadan kalkar. Bedeli `Rental.getCharge()` iki kez hesaplanmış olur: bir kez makbuz satırı için, bir kez toplam için. Hoca bunun önemsiz olduğunu söyledi: bir kişi 20, hatta 100 film kiralasa bile nispeten basit bir metodun 100 ya da 200 kez çağrılması arasındaki farkı hissetmeyiz. `fowler_04`'te `thisAmount`'un tanımı ve ataması silinmiş, kullanıldığı iki yere `aRental.getCharge()` çağrısı yazılmıştır; başka değişiklik yoktur. Testler geçti.

Bu, önceki derste görülen Replace Temp with Query eylemidir.

8.8 Adım 5: bonus hesabının `Rental`'a taşınması

`statement()` hâlâ uzundur ve içinde yorumlarla ayrılmış farklı alt işler vardır; her yorum ayrı bir alt iştir. Bedel hesabı `Rental`'a taşındı; şimdi gözü bonus hesabı çeker. Bonus hesabı da kira

kaydının filmini alıp fiyat koduna ve kaç gün kiralandığına bakar. Bedel hesabından hiçbir farkı yoktur: kullanılan bilgilerin hiçbiri `Customer`'a ait değildir, hepsi `Rental` ve `Movie`'ye aittir. Bu kez ara adım tutulmadan doğrudan Extract Method yapılıp Move Method ile `Rental`'a taşınır.

Hoca ayrıca adı da sadeleştirdi: `frequentRenterPoints` uzun bir addır. "Bonus" bankacılıkta da kullanılan, daha eskiden video oyunlarındaki ek puandan, ondan da önce ikramiye anlamından gelen bir terimdir; hem söylemesi hem yazması daha kolay, hem de daha anlaşılırdır. Bu yüzden `Rental`'a `getCharge()`'ın yanına `getBonus()` eklenir. `statement()` içindeki kümülatif bonus toplamı ise kaldırılamaz.

Ara adımdaki `Rental.getBonus()` (ekrandan, 01:55:30):

```
public int getBonus( ) {
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && getDaysRented() > 1)
        return 2;
    else
        return 1;
}
```

Hoca burada mantığı da sadeleştirdi. Eski kod önce puanı koşulsuz bir artırıyor, sonra yeni film bir günden uzun kiralandıysa bir daha artırıyordu. Yeni hâl: koşul tutuyorsa 2 puan, tutmuyorsa 1 puan. `statement()` içinde bonus artık `bonus += aRental.getBonus();` ile toplanır (ekrandan, 02:20:00). Hoca, önceki pakette bir metodun erişim belirtecinin yanlışlıkla paket düzeyinde kaldığını fark etti ve public olmasının yeterli olduğunu söyledi. Testler geçti.

8.9 Adım 6: toplamların ayrı metotlara çıkarılması

Aradan sonra hoca bu beşinci müdahaleyi özetledi ve `statement()`'ı biraz daha kurcaladı. `totalAmount` ve `bonus` kümülatif toplam oldukları için kaldırılamaz gibi görünür. Ama Replace Temp with Query eylemi hatırlanırsa bu geçici değişkenler de `statement()`'tan çıkarılabilir: toplam bedelin ve toplam bonusun hesabı, `statement()`'ın altında yapılan ayrı ek işler olarak düşünülebilir.

Varılacak nokta: `Customer`'a `getTotalCharge()` ve `getTotalBonus()` metotları eklenir. Bunlar da işlerini `Rental`'ın `getCharge()` ve `getBonus()` metotlarını kullanarak yapar. Daha önce `statement()` içinde olan geçici değişkenler bu metotlara taşınır.

`fowler_05` ile `fowler_06` karşılaştırması (ekrandan, 02:17:20): eskiden `statement()`'ın döngüsünde `totalAmount += aRental.getCharge();` vardı ve sondaki "Amount owed is" ile "You earned ... frequent renter points" satırları bu geçici değişkenlerden yazılıyordu. Şimdi döngüde bu toplamalar yoktur; son satırlar ilgili metotlar çağrılarak yazılır. Yeni metotlar:

```
public double getTotalCharge( ) {
    double result = 0;
    for( Rental each : _rentals )
        result += each.getCharge();
    return result;
}
public int getTotalBonus( ) {
    int result = 0;
    for( Rental each : _rentals )
        result += each.getBonus( );
    return result;
}
```

Bu metotların da içinde birer geçici değişken vardır, ama artık her biri yalnızca o odaklanmış alt toplam işine aittir; metotlar kısa ve tek işe odaklıdır. Kira listesi bu kez toplam bedel ve toplam bonus için ayrı ayrı dolaşılır. Hoca yine başarım kaygısını yanıtladı: bir dizinin üzerinde 100 kez ilerlemekle 200 kez ilerlemek arasında hissedilebilir bir fark olmaz; bu yüzden bu eylem çekinmeden yapılabilir. Testler geçti. Hoca bu noktada sıralama şemalarını her adımda çizmeyeceğini, farkın belirginleşeceği yerde göstereceğini söyledi.

8.10 Adım 7: ücret hesabının Movie'ye taşınması

`statement()` neredeyse yarı yarıya kısaldı. `Rental`'a epey şey eklendi; şimdi `Rental.getCharge()` göze batmaya başlar. Kendi metotlarına ve ilişkide olduğu filmin metotlarına erişmesinde normalde sakınca yoktur; ama `Movie`'nin metotlarına case'lerin içinde erişmek işi riskli hâle getirir. Yeni bir film türü eklenmek istense hem `Movie`'ye yeni bir tür sabiti, hem de `Rental`'daki switch'e yeni bir case eklemek gerekir. `Movie` türüyle ilgili bir değişikliğin `Rental`'a yansıması iyi bir şey değildir. Bu yüzden switch-case `Movie`'ye taşınırsa yan etkisi azalır.

Bedel hesabı için iki bilgi gerekir: kiralama süresi (`Rental`'da) ve film türü (`Movie`'de). Switch `Movie`'ye taşınır ve `Movie`'ye bilmediği kira süresi parametre olarak verilir. Yani `Rental`, `Movie`'ye "seni şu kadar gün kiraladılar, bundan doğacak bedeli söyle" der; `Movie` kendi türüne switch içinde bakar ve sonucu döndürür. Böylece switch-case en az dezavantajlı yere taşınmış olur ve `Movie` de basit bir getter-setter sınıfı olmaktan çıkıp kendi iş mantığını içeren bir sınıf olur. Çağrı zinciri şöyledir: `Customer.statement()` → `Rental.getCharge()` → `Movie.getCharge(daysRented)`; sonuç geri döner, `Customer` toplamda biriktirir.

`fowler_07`'de `Rental.getCharge()` tek satırlık bir iletime dönmüştür: `_movie.getCharge(_daysRented)`. `Movie`'ye eklenen metot (ekrandan, 02:30:00):

```
public double getCharge( int daysRented ) {
    double result = 0;
    switch ( getPriceCode() ) {
    case Movie.REGULAR:
        result += 2;
        if (daysRented > 2)
            result += (daysRented - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
    ...
```

Kod öncekiyle hemen hemen aynıdır: önce bir temel ücret eklenir, belli günden fazlaysa ek gün ücreti eklenir. Fark, gün sayısının artık parametreden gelmesi ve case anahtarlarının `Movie`'nin kendi sabitleri olmasıdır. Hoca, `Movie`'nin kendi alanına getter yerine doğrudan erişebileceğini, getter kullanmanın da sakıncası olmadığını söyledi. Artık yeni bir film türü eklenirken değişiklik yalnız `Movie` ile sınırlı kalır: yeni bir tür adı ve numarası ile yeni bir case eklenir; diğer sınıflara hiçbir yan etkisi olmaz. Switch-case'in zararı en aza indirilmiştir. Testler geçti.

8.11 Adım 8: bonus hesabının Movie'ye taşınması

Aynı iş bonus için yapılır (`fowler_08`): `Rental.getBonus()` filme "şu kadar gün kiralandım, ne kadar bonus kazandıracaksın?" diye sorar. `Movie.getBonus(int daysRented)` aynı mantıkla çalışır: fiyat kodu yeni film ve kira 1 günden uzunsa 2 puan, aksi hâlde 1 puan. Metot `Rental`'dan tamamen kesilmemiş, `Movie`'ye iletilmiştir; ama bu basit bir iletim değil, sonucu alıp kullanan bir iletimdir. Testler geçti.

8.12 Adım 9: Customer'ın Movie'ye doğrudan erişiminin kaldırılması

Bu değişikliklerden sonra sıralama şemasına bakıldığında hep `Customer` `Rental`'la, `Rental` da `Movie`'yle konuşur. Tek bir istisna kalmıştır: `Customer` makbuzda filmin adını yazmak için doğrudan `Movie`'ye erişir (`aRental.getMovie().getTitle()`). `Customer` artık filmin üzerinden başka hiçbir işlem yapmadığına göre bu erişime gerek yoktur; `Customer` `Rental`'a "ilişkili olduğun filmin adı ne?" diye sorabilir.

`Rental`'a `getMovieTitle()` eklenir (ekrandan, 02:40:00); `Customer.statement()` bu metodu çağırır, `Rental` da filmine sorup cevabı döndürür. `Rental`'ın son hâli:

```
public double getCharge( ) {
    return _movie.getCharge( _daysRented );
}
public int getBonus( ) {
    return _movie.getBonus( _daysRented );
}
```

```
public String getMovieTitle( ) {
    return _movie.getTitle( );
}
```

Böylece tam bir katmanlama elde edilir: `Customer` yalnız `Rental` ile, `Rental` yalnız `Movie` ile ilişkilidir; dolaylı bağlaşımlar kaldırılmış, metot çağrıları yalnız doğrudan bağlaşımlar üzerinden yapılır hâle gelmiştir. Katmanlı tasarımın genel avantajı, bir katmanda yapılan değişikliğin yalnız ilişkili olduğu katmanı etkilemesi, diğerlerini etkilememesidir. `Movie`'de değişiklik gerekmedi, çünkü `getTitle()` zaten vardı. Testler geçti.

8.13 Kalan iş ve kodun genel durumu

Hoca son hâli birlikte gözden geçirdi. `statement()` içinde makbuzun sonuna footer ekleme işi hâlâ ayrı bir alt iştir; bunun "add footer" gibi ayrı bir metoda çıkarılması iyi olur. Hoca bunu hiçbir pakette yapmadığını, "borç" olarak bıraktığını söyledi. Bunun dışında `Customer` çok uzun olmayan metotlar içerir, hep kendi üye alanları üzerinden çalışır ve en fazla bir adım ötedeki ilişkili nesneye bir şey sorup cevabını alır. `Rental` aradaki adımlarda biraz uzasa da hiçbir zaman aşırı şişman olmadı ve sonunda epey hafif (lightweight) bir sınıf hâline geldi. Geriye yalnız `Movie`'deki switch-case kaldı; bunun için iki alternatif 10. bölümde anlatılmıştır.

9. Kara Kutu ve Beyaz Kutu Sınaması

Hoca, yazılım sınamasının başlı başına bir konu olduğunu, ama burada yürütülüş şekline göre bir ayrım yapmak istediğini söyledi. Sınanacak şey bir elektronik devre, bir makine ya da bizim meslekte bir sınıf kodu olabilir.

Kara kutu sınaması (black box testing): sınanacak şeyin iç çalışma mantığı bilinmiyorsa, ona erişilemiyor ya da görülemiyorsa yapılır. Kutu şeffaf değildir, açılıp bakılamaz. Test, yapması gereken işler üzerinden yapılır: verilen geçerli girdilere karşılık doğru sonuç elde ediliyor mu? Nispeten kolaydır, çünkü yapılacak tek şey budur. Zorluğu, kaç farklı girdi verilince içimizin rahat edeceğidir. Bunun için çeşitli yöntemler vardır; girdi grupları öngörülebiliyorsa her gruptan bir girdiyle yapılan test en doyurucu testtir. Hocanın örneği: yaş aralıklarına göre bir insanın sahip olması gereken kan basıncının hesaplanması. Tıp literatüründe yaş grupları bellidir; her gruptan bir örnek alıp hesaplatınca doğru çıktı alınıyorsa yeterli sınama yapılmış sayılır.

Beyaz kutu sınaması (white box testing): sınanacak birimin iç işleyişi bilinir ve test case'ler buna göre ince ayarlanır. Hedef mümkün olan en yüksek kod kapsamıdır (code coverage). Kaynak kod bilindiği için bütün if dallanmalarının hangi durumlarda gerçekleşeceği, döngülerin hangi değerlerden hangi değerlere döneceği bilinir; bütün olası konumlardan geçmeyi sağlayacak farklı girdiler verilerek kara kutuyla ulaşılabilecek olandan daha kapsamlı bir sınama yapılır. Avantajı budur; dezavantajı daha fazla iş gerektirmesidir.

Yazılım sınaması maliyetli olduğu için projelerde sınama etkinliklerine belli bir zaman ya da iş gücü oranı ayrılır ve sınama ona sığdırılır. Bu yüzden her yerde beyaz kutu yapmak gerekmez: sistemin kritik ve karmaşık parçaları için beyaz kutu, daha basit kalan bölümler için kara kutu sınaması genelde yeterlidir.

Örnekle bağlantısı: metot taşıma sonucunda `Rental`'a yeni bir `getCharge()` metodu eklendi. Bunun için yeni bir test case yazmak beyaz kutu yaklaşımıyla devam etmek, mevcut testlerle devam etmek kara kutu yaklaşımıdır. Beyaz kutu test kalitesini artırır ama zaten sınırlı olan proje zamanını uzatabilir. Hoca işi basit tutmak için mevcut testlerle, kara kutu yaklaşımıyla devam etmeyi seçti.

10. Tip Kodunun Kaldırılması İçin İki Alternatif

`Movie`'deki switch-case şu an yan etkisi en az olacak yerdedir. Yine de "kafama koyduğumu düzelteceğim" denirse iki yol vardır.

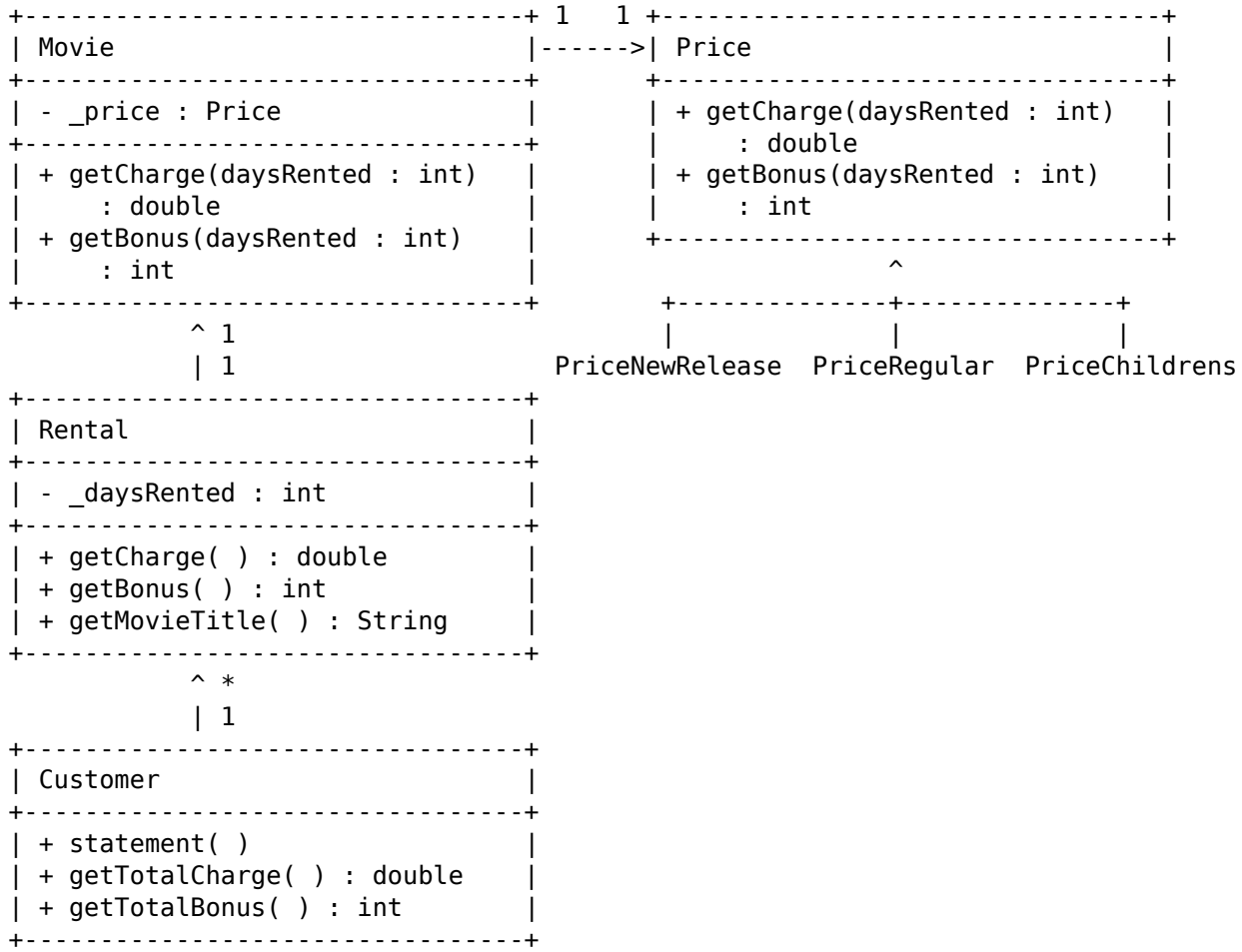
10.1 Replace Type Code with Subclasses

Üç farklı film türü olduğuna göre soyut bir `Movie` sınıfı ve onun somut alt sınıfları `Childrens`, `Regular`, `NewRelease` oluşturulabilir. Böylece `priceCode` üyesine gerek kalmaz; `getCharge()`'ın her case'i ilgili alt sınıfın `getCharge()` gövdesine gider. `Rental`, bu üç alt sınıftan birinin nesnesiyle ilişkili olur.

Ama bu çözüm üç yeni sınıf doğurur ve `Movie` soyut olduğu için test kodunda yapıldığı gibi `Movie` nesnesi oluşturulamaz; ayrı ayrı `NewRelease`, `Regular`, `Childrens` nesneleri oluşturmak gerekir. Kısacası dışarıya verilen arayüz değişir, test kodu da değişir ve geriye doğru uyumluluk kaybedilir. Bu, refactoring'i tam tanımıyla yapmamak demektir. Geriye doğru uyumluluk önemliyse bu değişiklik yapılmaz. Ama uyumluluk gerekmiyorsa, yazılım henüz müşterilere dağıtılmamışsa (sektördeki deyimle "canlıya alınmamışsa", hoca bu terimi pek sevmediğini söyledi), yani henüz bakım aşamasına geçilmemişse ve yinelemeli bir süreçte bir refactoring iterasyonu yapılıyorsa yapılabilir. Yazılım müşterilere dağıtılmışsa yapmamak da rahatlıkla savunulabilir.

10.2 Replace Type Code with Strategy

Hocanın tercih ettiği yol budur (`fowler_10_alt`, ekrandan, 02:46:40). Filmlerde değişen şey fiyat türü ve ondan kaynaklanan kira bedeli ile bonus hesaplarıdır. Bu yüzden bir `Price` soyut sınıfı oluşturulur; içinde `getCharge` ve `getBonus` soyut metotları tanımlanır. Somut alt sınıfları `PriceNewRelease`, `PriceRegular` ve `PriceChildrens`'tır. `Movie`'nin eski hâlindeki switch'lerin case'leri ilgili alt sınıflara taşınır.



Hoca kavramsal model açısından alt sınıf çözümünün daha sade olduğunu kabul etti: problem alanında "şu tür filmler var" diye anlatılır. Strateji çözümünde bir alt kırılım fazla yapılmış olur. Ama ona göre bu bir sorun değildir:

> **Hocanın vurgusu:** İyi bir tasarım, problem hangi düzeyde parçalara ayrılıyorsa o düzeyde parçalara ayrılan tasarımdır. Daha da alt parçalara ayrılırsa gereksiz kar-

maşıklık oluşur.

Buradaki fazladan kırılım gereksiz karmaşıklığa yol açmaz; tersine, daha az kod kusuru içeren bir tasarım ortaya çıkar. Üstelik bu değişiklikte `Movie`'nin dış arayüzü korunduğu için test case'leri değiştirmek gerekmez. Hoca bu yüzden bu çözümü tercih etti.

11. Anti Patterns (Karşıt Kalıplar)

Bütüncül refactoring örneği bitince hoca kısaca karşıt kalıplara değindi; konunun müfredatta olmadığını, ama varlığının bilinmesini istedi. Slayttaki metin (ekrandan, 02:50:00):

- **Antipattern nedir?**
  - Karşıt kalıp çözüm bir tasarım kalıbı gibi görünür ancak öyle değildir.
  - Karşıt kalıplar aslında tasarım kalıplarının karşıtları olarak yazılım geliştirme sürecinde tekrar eden bazı yanlışları ifade ederler.
  - Karşıt kalıpları bilmek, yazılım geliştirme sürecinde karşılaşılabilecek ciddi problemleri önceden tahmin edebilmeyi ve tedbir almayı kolaylaştırır.
- **Karşıt kalıp ile kod kusuru farkı:**
  - Karşıt kalıplar tasarım düzeyine, kod kusurları gerçekleme düzeyine daha yakındır.
  - Karşıt kalıplar yazılım yaşam döngüsünün diğer evreleri ile de ilişkilidir.
- **Karşıt kalıp türleri** (henüz müfredata eklenmedi): yazılım geliştirme karşıt kalıpları, yazılım mimarisi karşıt kalıpları, yazılım proje yönetimi karşıt kalıpları.

Hoca farkı şöyle açıkladı: kod kusurlarına bakıldığında, kusur ya tek bir sınıfta ya da en fazla bir iki adım ötedeki ilişkili sınıfa uzanan bir yerdeydi. Bu yüzden Extract ve Move eylemleriyle aynı sınıfa ya da ilişkili veya kalıtımla bağlı bir sınıfa taşıma yapılabiliyordu. Kod kusurları genelde metot, sınıf, en fazla bir iki sınıf düzeyindedir. Karşıt kalıplar ise kodla ilgiliyse birden fazla sınıfa yayılmış olgulardır; insanı iyi bir yere götürmeyen tasarım kalıplarıdır. Yalnız tasarım ve modelleme değil, proje yönetimi düzeyinde tanımlanmış karşıt kalıplar da vardır.

Hoca konunun klasik bir kitabı ve bir web sitesi olduğunu, oradan incelenebileceğini söyledi. Web sitesinde refactoring için de bir bölüm olduğunu ve orada kod kusurlarının farklı gruplandığını gösterdi: örneğin Long Method, Large Class ve Primitive Obsession "bloaters" (bir şeyi hantallaştıran, aşırı şişiren) grubundadır; switch statements gibi kusurlar başka bir gruptadır.

> **Ek bilgi (derste anlatılmadı):** Hocanın gösterdiği site büyük olasılıkla SourceMaking'dir (altyazıda "sauce making" olarak geçiyor); bu sitede tasarım kalıpları, karşıt kalıplar ve refactoring bölümleri vardır. Konunun klasik kitabı büyük olasılıkla Brown, Malveau, McCormick ve Mowbray'in *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis* (1998) kitabıdır; geliştirme, mimari ve proje yönetimi ayrımı bu kitaptan gelir.

12. Örnek Sorular

Dersin sonunda hoca ikinci ara sınavın sorularını açtı ve öğrencilerin verdiği cevaplar üzerinden kalıpları tartıştı. Aşağıda soruların metni ekrandan, çözüm tartışması hocanın anlatımından alınmıştır.

Soru 1 (ekrandan, 03:00:00): Uçtan uca güvenli bir haberleşme sağlayacak bir chat programı hazırlıyorsunuz. Bu programda birden fazla kripto algoritmasının kullanılabilmesini istiyorsunuz. Hangi tasarım kalıbını neden kullanırsınız? Yukarıdaki açıklama kapsamında tasarımınızı yapınız.

> **Çözüm (derste verilen):** İki kişi arasında güvenli bir iletişim kanalı kurulacak ve trafik şifrelenecektir. Literatürde DES gibi blok tabanlı, yerine koyma (substitution) tabanlı, eliptik eğri gibi asimetrik anahtarlı çeşitli algoritmalar vardır. Sistemde birden fazla algoritma kodlanabilir, ama bir haberleşme belirli bir anda tek bir algoritmayla şifrelenir; bir paketin bir kısmı bir yöntemle, kalanı başka yöntemle şifrelenmez. Bu yüzden doğru kalıp **Strategy**'dir. Mesaj gönderen taraf (`sendMessage(plainText)`) metni doğrudan göndermez, ilişkili olduğu şifreleyiciye şifreletir. Ortak bir `Cipher` arayüzü (ya da soyut sınıfı) bir şifreleme metodu tanımlar: dizgiyi alır, şifreler, şifreli hâlini döndürür. `DESCipher`, `RSACipher` gibi somut sınıflar bunu kendi yöntemleriyle gerçekler (kimi iki anahtar, kimi tek anahtar

ister). Sistem mutlaka bir `Cipher` nesnesiyle ilişkilidir; DES verilirse DES ile, daha fazla güvenlik gerekirse RSA verilerek RSA ile şifreler.

State demek, bu açıklamayla doğru değildir. Ama cevapta örneğin "haberleşme bant genişliği azalınca ya da artınca daha hızlı veya daha yavaş çalışan bir algoritmaya geçilebilir" gibi bir durum geçişi açıklanırsa yapı State olarak da yorumlanabilir. İki kalıbın sınıf şeması aynıdır; fark, State'te bir durumdan başkasına geçiş olması, Strategy'de ise genelde bir kez seçilen stratejiyle bütün iş boyunca çalışılmasıdır. Decorator uygun değildir: Decorator'ın amacı bir varlığa sarmalayarak yeni yetenekler eklemektir; soruda böyle bir istek yoktur. Chain of Responsibility de uygun değildir (Soru 2 tartışmasına bakınız).

Soru 2 (ekrandan, 03:00:00): Observer ve Mediator tasarım kalıplarının benzer ve farklı yönlerini kısaca açıklayınız.

Çözüm (derste verilen): Benzer yön: her ikisinde de birbiriyle iletişimde olan varlıklar vardır. Farklı yön: Mediator'da nesneler doğrudan değil, ortak bir ara bulucu (mediator) üzerinden dolaylı haberleşir; bir nesne mediator üzerinden başka bir nesneyi etkileyebilir ya da ona iş yaptırabilir. Observer'da ise izlenen (subject) ve izleyiciler (observer) vardır; çalışma, durum değişikliklerinin gözlenmesi ve işlenmesi biçimindedir.

Bu tartışma sırasında bir öğrencinin Soru 1 için önerdiği Chain of Responsibility (CoR) de konuşuldu. Hoca kalıbı şöyle anlattı: birbiriyle ilişkili bir varlıklar silsilesi vardır ve bir işin yapılması istenir. Silsiledeki bir varlıktan iş istenir; kendisinde bu işi yapacak bilgi, beceri, veri ya da yetenek varsa yapar ve sonucu döndürür; yoksa ilişkide olduğu bir sonraki nesneye sorar. Banka örneği: sigorta yaptırmak için gişeye gidilir; gişe çalışanı bu konuda yeterli bilgiye sahip değildir ama bankanın bu yeteneği vardır; işi sigorta işlemleri uzmanına yönlendirir; uzman da çok ilginç bir durumda şube müdürüne sorar ve cevap geri gelir. Üniversitede de bir talep, inisiyatifinde olmayan birimden bölüm başkanlığına, oradan dekanlığa iletilip yetkili yerde (örneğin yönetmelikle ilgili bir sorunsa dekanlıkta) çözülür. Chat programında ise şifreleme algoritmaları birbirine "sen bunu şifreleyebiliyor musun, edemiyorsan sonrakine ilet" diye bağlı değildir; algoritma o anda seçilip verilir. Bu yüzden CoR bu senaryoya uymaz.

Soru 3 (ekrandan, 03:00:00): Bağlaşımın (coupling) neden sıkı/yüksek (tight/high) olması istenmez? Bağlaşımı azaltmaya yönelik bir tasarım kalıbının adını vererek bu kalıbı birkaç cümle ve bir UML sınıf şeması ile kısaca anlatınız. Seçeceğiniz kalıp Soru 4'teki ile aynı olmamalıdır.

Çözüm (derste verilen): Hedef hep düşük bağlaşım, yüksek uyumdur. Yüksek bağlaşım, bir yerde yapılan değişikliğin başka yerlerde ek yan etkilere ve hatalara yol açmasına neden olabilir; ayrıca yazılımın yeniden kullanım oranını düşürür. Bu konular dersin ilk bölümlerinde tartışılmıştı; birkaçını yazmak yeterlidir.

İkinci kısım için dikkat: her tasarım kalıbı bağlaşımı düşürmez. Hoca, lisans öğrencilerine yaptırdığı bir çalışmayı anlattı: aynı problem önce ilk akla gelen biçimde, sonra nesne yönelimi doğru kullanılarak, en sonunda da bir tasarım kalıbıyla çözüldü ve ölçüldü; kalıplar her zaman bağlaşımı düşürmedi, bazen biraz yükseltti. Ders notlarındaki basit çözüm, normal çözüm ve kalıplı çözüm karşılaştırmalarında da kalıplı çözümde yeni sınıflar ve yeni ilişkiler ortaya çıktığı görülür. Bağlaşımın asgarinin altına inmesi de istenmez (hoca bunu karnı doyurmak için gereken asgari yemek benzetmesiyle anlattı). Dörtlü Çete'nin (GoF) kitabında özellikle bağlaşımı azaltan kalıplar olarak **Facade, Chain of Responsibility, Command, Observer ve Mediator** geçer. MVC de bu soru için uygun bir cevaptır. Abstract Factory gibi kalıplar yeni sınıflar ve bağlaşımlar getirse de problem alanını oluşturan varlıklar açısından kısmi bir bağlaşım iyileştirmesi sağlayabilir; bu listede olmayan bir kalıp seçilip açıklama ve sınıf şemasıyla iyi gerekçelendirilirse kabul edilebilir. Problem alanındaki varlıkların ilişkilerindeki karmaşıklığı toparlayan her kalıp bunu yapar.

Soru 4 (ekrandan, 03:00:00): Nesneye yönelimli tasarımda switch-case komutlarından mümkün olduğunca kaçınılması istenir. Bu yönde bize yardımcı olabilecek bir tasarım kalıbının adını vererek az önce değinilen isteğimize ulaşmada bize nasıl yardımcı olabileceğini kısaca anlatınız. Seçeceğiniz kalıp Soru 3'teki ile aynı olmamalıdır.

Çözüm (derste verilen): Kalıtım ve çok biçimliliği kullanan herhangi bir kalıp se-

çilip iyi anlatılırsa doğru cevaptır (örneğin bu derste görülen State ve Strategy: her case'in kodu ayrı bir alt sınıfa gider). Uygun olmayan cevaplar ve nedenleri:

- **Builder:** kalıbın kendi içinde kaçınılamayan bir switch-case vardır. Builder, bir parça zincirinden bir ürün oluşturur; her parçayı ayrı işleyip bütüne monte eder ve gelen parçanın hangisi olduğunu switch-case ile denetler. Bu switch daha iyi bir yere taşınabilir ama kalıp boyunca ondan kurtulmak mümkün değildir; ders notlarındaki tanımda da örneklerde de vardır.
- **MVC:** switch-case'ten kurtarma ile ilgisi yoktur; bağlaşımı düşürmeye yöneliktir. Soru 3 için uygun, Soru 4 için uygun değildir.
- **Prototype:** prototiplerden oluşturulan yeni türlerin bir kayıt defterine (registry) eklenmesi gerekir ve bu kısımda ister istemez bir switch-case oluşur; problem tanımında da düzeltilecek bir switch-case yoktur.
- **Singleton:** bir sınıftan tek bir nesne olmasıyla ilgilidir; switch-case ile ilgisi yoktur.
- **Adapter:** farklı bir metot arayüzünü uyarlamakla ilgilidir; uymaz.
- **Facade:** karmaşık bir sistemi küçük bir ayak iziyle (footprint) dışarıya açar; Soru 3'e yönelik bir çözümdür.
- **Flyweight, Proxy, Memento, Mediator:** switch-case ile pek ilgileri yoktur.

Dersin Toparlaması

Bu derste kod kusurlarının kataloğu tamamlandı. Large Class için Extract Class, Extract Subclass ve Duplicate Observed Data; Divergent Change ve Shotgun Surgery için Extract Class, Move Method ve Move Field; Primitive Obsession için "Replace" ile başlayan eylemler (Data Value with Object, Type Code with Class / Subclasses / State-Strategy, Array with Object); Feature Envy için Move Method ve Extract Method görüldü. Uyumun üye alan erişim deseniyle nasıl ölçülebildiği ve Extract Class'ın bu ölçüyü nasıl iyileştirdiği sayısal bir örnekle gösterildi.

Dersin ikinci yarısında bu eylemler Fowler'ın video kiralama örneğinde art arda uygulandı: switch bloğu önce `Customer` içinde bir metoda çıkarıldı, sonra `Rental`'a, en sonunda `Movie`'ye taşındı; geçici değişkenler sorgu metotlarıyla değiştirildi; `Customer`'ın `Movie`'ye doğrudan erişimi kaldırılarak Customer → Rental → Movie katmanlaması kuruldu. Her adımda test kodu değiştirilmeden testler yeniden çalıştırıldı; bu, refactoring'in dış arayüzü korumasının somut ölçütüdür. Kalan tip kodu için alt sınıf çözümünün geriye doğru uyumluluğu bozduğu, `Price` stratejisinin ise arayüzü koruduğu tartışıldı. Kod kusurlarının gerçekleme düzeyinde ve bir iki sınıfla sınırlı, karşıt kalıpların ise tasarım düzeyinde ve çok sınıflı olduğu belirtildi. Son olarak ara sınav soruları üzerinden State ile Strategy arasındaki fark, Observer ile Mediator karşılaştırması, Chain of Responsibility ve bağlaşımı azaltan ya da switch-case'ten kurtaran kalıplar tekrar edildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.